



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Topic: Backtracking

Theory: In many applications of the backtrack method, the desired solution is expressible as an n -tuple $(x_1, \dots, x_n)$, where the x_i are chosen from some finite set S_i . Often the problem to be solved calls for finding one vector that maximizes (or minimizes or satisfies) a *criterion function* $P(x_1, \dots, x_n)$. Sometimes it seeks all vectors that satisfy P . For example, sorting the array of integers in $a[1 : n]$ is a problem whose solution is expressible by an n -tuple, where x_i is the index in a of the i th smallest element. The criterion function P is the inequality $a[x_i] \leq a[x_{i+1}]$ for $1 \leq i < n$. The set S_i is finite and includes the integers 1 through n . Though sorting is not usually one of the problems solved by backtracking, it is one example of a familiar problem whose solution can be formulated as an n -tuple.

Control abstraction:

```
void Backtrack( int k )
```

```
// This is a schema that describes the backtracking process //using recursion. On entering, the first k-1 values x[1], x[2], //...., x[k-1] of the solution vector x[1:n] have been //assigned. x[] and n are global.
```

```
{
    for (each x[k] such that x[k] ∈ T(x[1], ..., x[k-1]))
    {
        if (Bk (x[1], x[2], ..., x[k]))
        {
            if (x[1], x[2], ..., x[k] is a path to an answer node)
                output x[1:k];

            if (k < n) Backtrack(k+1);
        }
    }
}
```



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Batch: B2	Roll No.: 16010124107
Experiment No. 8	
Grade: AA / AB / BB / BC / CC / CD / DD	
Signature of the Staff In-charge with date	

Title: Study, Implementation, and Analysis of Sum of Subset Problem.

Objective: To learn the Backtracking strategy of problem solving for Sum of subset

CO to be achieved:

CO 2 Analyze and solve problems for divide and conquer strategy, greedy method, dynamic programming approach and backtracking and branch & bound policies.

Books/ Journals/ Websites referred:

1. Ellis horowitz, Sarataj Sahni, S.Rajsekaran," Fundamentals of computer algorithm", University Press
2. T.H.Cormen ,C.E.Leiserson,R.L.Rivest and C.Stein," Introduction to algortihtms",2nd Edition ,MIT press/McGraw Hill,2001

Pre Lab/ Prior Concepts:

Data structures, Concepts of algorithm analysis

Historical Profile:

Subset sum problem is to find subset of elements that are selected from a given set whose sum adds up to a given number K. We are considering the set contains non-negative values. It is assumed that the input set is unique (no duplicates are presented).

One way to find subsets that sum to K is to consider all possible subsets. A power set contains all those subsets generated from a given set. The size of such a power set is 2^N .

Input:

A vector $X=\{x_1, x_2 \dots x_n\}$ for all n elements in the set where $X_i=0$ (element not added) or $x_i=1$ (element added in the solution tuple).

Output:

Summation of the chosen numbers must be equal to given number M and one number can be used only once.



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

BACKTRACKING CONDITION

$$B_k(x_1, \dots, x_k) = \text{true} \text{ iff } \sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i \geq m$$
$$\text{and } \sum_{i=1}^k w_i x_i + w_{k+1} \leq m$$

New Concepts to be learned:

Application of algorithmic design strategy to any problem, Backtracking method of problem solving Vs other methods of problem solving problem sum of subset and its applications.

Algorithm:

Algorithm sumOfSub(s, k, r)

```
{  
    //It is assumed w[1]<=m and Sigma(i=1 to m)w[i]>=m  
  
    //generate the left child. Note: s+w(k)<=M since Bk-1 is true.  
  
    X[k]=1;  
  
    if (S+W[k]=m) then write(X[1:k]);  
    //Subset found. there is no recursive call here as  
    W[j]>0, 1<=j<=n.  
  
    else if (S+W[k]+W[k+1]<=m)  
        then sumOfSub(S+W[k], k+1, r- W[k]);    //moving to next sub-problem.
```

[Similarly, assume the array is presorted and we found one subset. We can generate next node excluding the present node only when inclusion of next node satisfies the constraints.]

```
if ((S+ r- W[k]>=m)and (S+ W[k+1]<=m)) then //generate right  
    {  
        //child and those satisfying 2 bounding functions  
  
        X[k]=0;
```



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

```
        sumOfSub (S, k+1, r- W[k]);  
    }  
}
```

Implementation(Code):

```
#include <bits/stdc++.h>  
using namespace std;  
#define int long long  
#define yes cout << "YES" << endl;  
#define no cout << "NO" << endl;  
#define input for(int &i:v) cin >> i;  
#define vall v.begin(),v.end()  
  
void print(const vector<int>& v) {  
    for (int i : v) cout << i << " ";  
    cout << endl;  
}  
  
void solve(vector<int>& arr, int idx, int target, int curr, vector<int>&  
subset, int& count) {  
    if (curr == target) {  
        count++;  
        cout << "Solution " << count << ": { ";  
        for (int x : subset) cout << x << " ";  
        cout << "}\n";  
        return;  
    }  
    if (idx == (int)arr.size() || curr > target) return;  
  
    subset.push_back(arr[idx]);  
    solve(arr, idx + 1, target, curr + arr[idx], subset, count);  
    subset.pop_back();  
  
    solve(arr, idx + 1, target, curr, subset, count);  
}  
  
void solve() {  
    int n, target;
```



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

```
cin >> n >> target;

vector<int> arr(n);
for (int& x : arr) cin >> x;

sort(arr.begin(), arr.end());

vector<int> subset;
int count = 0;
solve(arr, 0, target, 0, subset, count);

if (count == 0) cout << "No solution found\n";
else cout << "Total solutions: " << count << "\n";
}

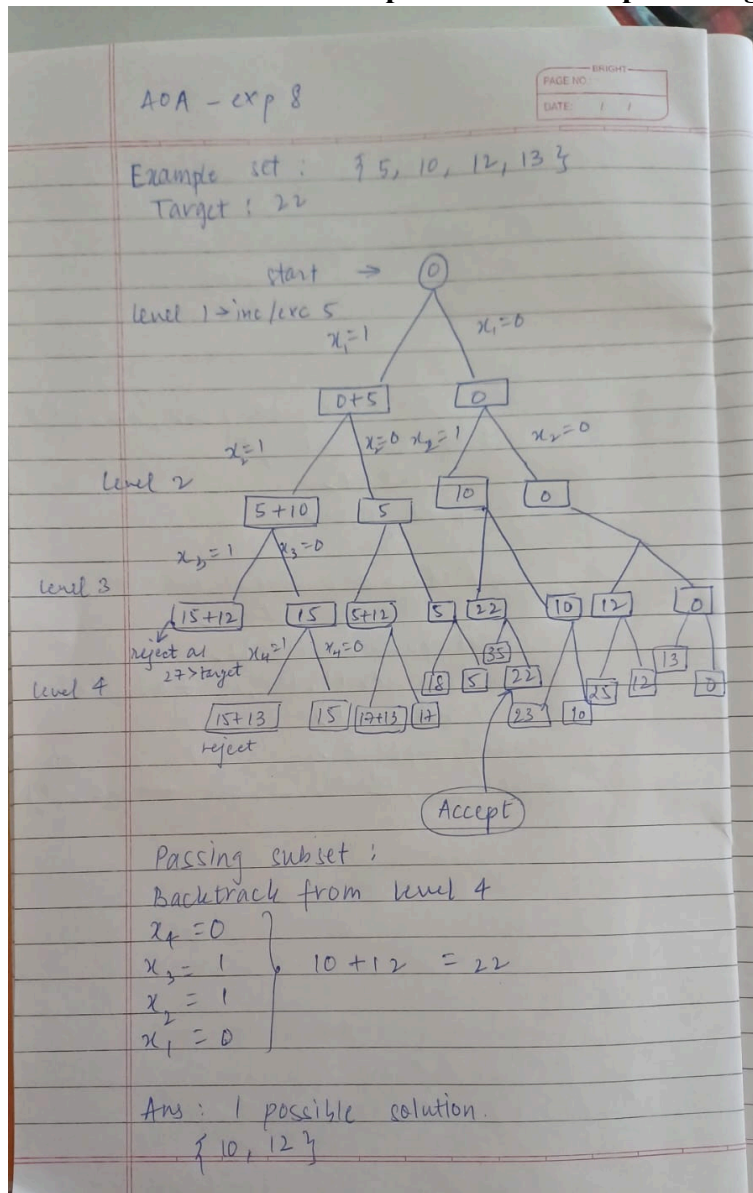
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
}
```

Output:

```
er 2\Analaysis of Algorithms
ile }
10 8
1 2 3 4 5 6 7 8 9 10
Solution 1: { 1 2 5 }
Solution 2: { 1 3 4 }
Solution 3: { 1 7 }
Solution 4: { 2 6 }
Solution 5: { 3 5 }
Solution 6: { 8 }
Total solutions: 6
D:\C:\Users\evade\Music\CVN\
```

Example sum of subset Problem along with state space tree:

K. J. Somaia College of Engineering, Mumbai-77
(A Constituent College of Somaia Vidyavihar University)
Department of Computer Engineering



Analysis of Backtracking solution for :

Backtracking in subset sum tries all combinations.
At each element, two choices exist.
Include the element or exclude it.

So state space is a binary tree.
Height of tree = n elements.
Total nodes $\approx 2^n$.

Time complexity is exponential.
Worst case = $O(2^n)$.
Because all subsets are explored.



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Pruning reduces unnecessary work.

If current sum > target → stop branch.

If remaining elements cannot reach target → stop.

Even with pruning, worst case stays exponential.

Space complexity depends on recursion.

Maximum depth = n .

So space = $O(n)$.

Conclusion:

In conclusion, the backtracking approach for the subset sum problem is simple and systematic, as it explores all possible subsets by making include or exclude decisions at each step. It ensures that no possible solution is missed, which makes it correct and complete. However, this method has an exponential time complexity in the worst case, since it may need to examine all 2^n subsets. Although pruning techniques help reduce unnecessary exploration and improve practical performance, they do not change the overall worst-case complexity. The space complexity remains linear due to the recursion stack. Therefore, backtracking is suitable for smaller input sizes but becomes inefficient for large datasets due to its high computational cost.